# BetterRhythm: Architecture Proposal

**Charles Montluc** · A Telegram schedule planner that learns how I work
**Stack:** n8n, Telegram, OpenRouter, Google Sheets, Open-Meteo, JavaScript · **Target:** Level 3

## 1. The Need

Busy students and young professionals tend to overload their days because they plan by gut feeling, and nothing learns from how the day actually went. They usually rely on a notes app or calendar blocks, and when a meeting slips they re-plan by hand or give up; nothing records how long tasks really took or when they focus best. BetterRhythm is a Telegram bot: the user describes their day in free text and receives a time-blocked schedule based on their own history and the weather. Each evening report refines the next plan.

## 2. The Trigger

The workflow starts when I message my own Telegram bot, so I can run it at any time from my phone. To debug, I pin the trigger payload in n8n and replay it, and I seed five days of history in the sheet so the learned profile is already visible during the demo.

| Command | Example I type | Reply in the same chat |
|---|---|---|
| /plan | /plan finish report, dentist 15:00, gym, slept badly | Time-blocked day with the trade-offs explained |
| /replan | /replan dentist moved to 17:00 | Rebuilt schedule showing what was compressed or dropped |
| /done | /done report 9-12, gym skipped | What the bot learned (focus hours, time factor) |
| /help | /help | List of commands with one example each |

## 3. Architecture & Data Flow

Step numbers match the tags on the diagram (page 2). Each step says what it passes on.

1. **Telegram Trigger** → Outputs message text, chat id and sender to step 2.
2. **Parse Command** (Code, JavaScript) → Splits the command from the free text, checks the strict /done format and prepares a usage message for unknown commands. Passes {cmd, text, chat_id} on.
3. **Load Profile** (Sub-workflow A) → Reads the sheet and returns peak hours, time factor, recent energy and the last saved plan (defaults when there is no history).
4. **Command Router** (Switch) → Routes on the command:
   - /plan, /replan → steps 5 to 10.
   - /done → 4a Update Profile (Code) compares planned and actual times and recomputes peak hours and time factor; 4b Save Profile and Signals (Sheets); then step 11.
   - /help or anything else → the usage text from step 2, then step 11.
5. **Fetch Weather** (HTTP Request) → Calls Open-Meteo for the hourly forecast; 3 retries with backoff, and the plan continues without weather if it still fails.
6. **Extract** (AI Chain 1) → Turns the free text into JSON: tasks, fixed events, energy, constraints.
7. **Validate** (Set) → Builds a clean array; invalid JSON falls back to defaults (30 min per task, normal energy).
8. **Build Schedule** (Code) → Places fixed events first, then deep work in peak hours and light tasks in the remaining gaps, with durations adjusted by the time factor and energy level; outdoor tasks avoid rain. Flags any conflict. Passes {schedule[], dropped[], conflict} on.
9. **Write Plan** (AI Chain 2) → Writes the HTML message and explains the trade-offs and any conflict; instructed not to comment on health or mood.
10. **Save Plan** (Google Sheets) → Appends the plan and the extracted signals.

11. **Send Telegram Message** → Sends {chat_id, html} back to the same chat.
12. **Error Logger** (Sub-workflow B) → Any node still failing after 3 retries sends {node, error, chat_id} here; it writes the errors tab and sends a fallback reply. It runs as its own workflow.

## 4. Level Aim & Cut Line

Targeting Level 3: two AI chains (Extractor and Writer) around a Code scheduler, two sub-workflows, and a profile that learns from history. If time runs short, I will cut in this order:

| # | Cut |
|---|---|
| 1 | The /replan command (re-sending /plan with the change gives the same result) |
| 2 | Per-category time factors become one global factor |
| 3 | Energy tracking from phrases such as "slept badly" (the scheduler keeps peak hours and time factor) |
| 4 | Last resort: both chains become one AI Agent calling Sub-workflow A as a tool (Level 2) |

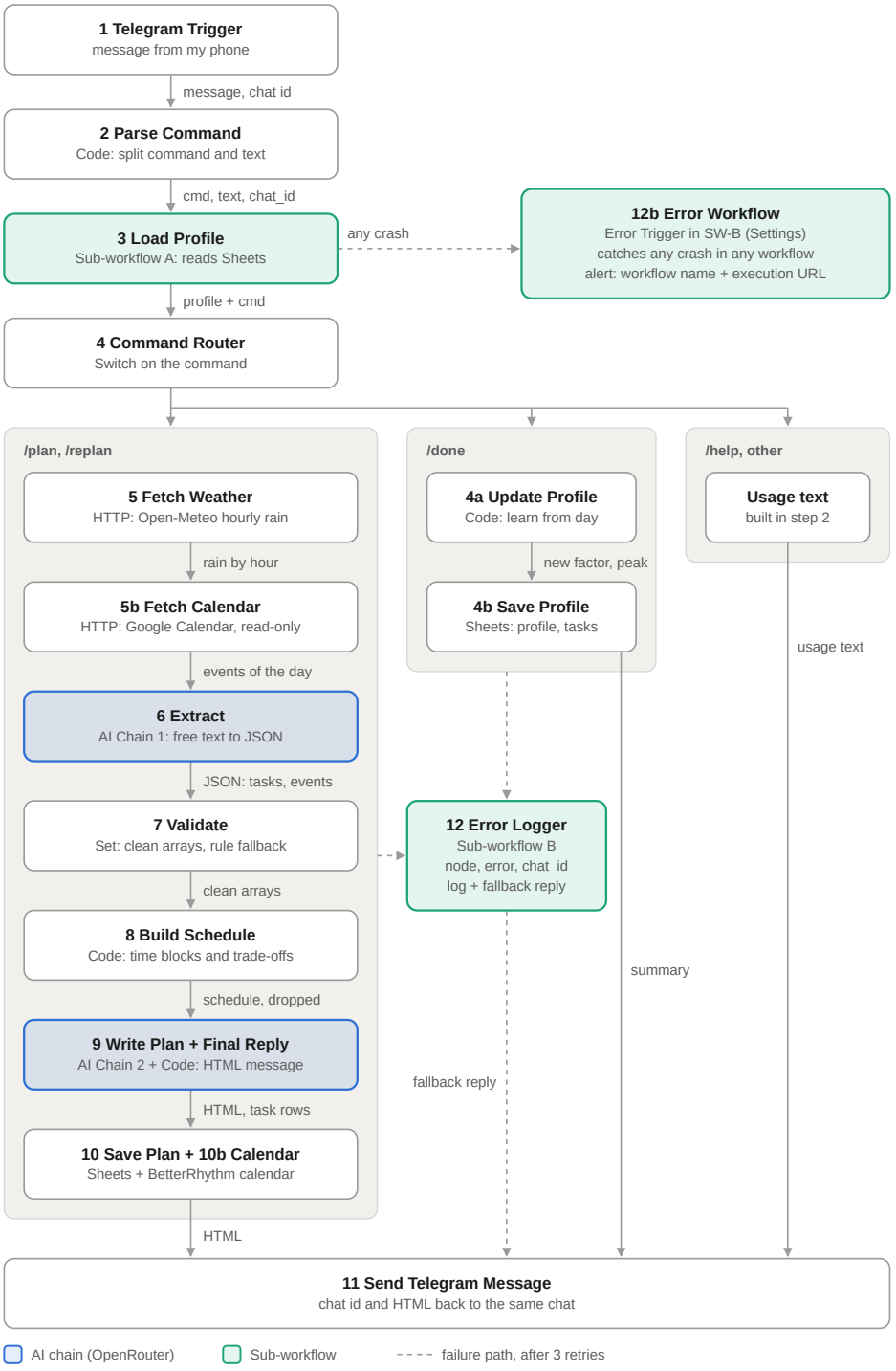
The core loop /plan, /done, changed next plan, and the Open-Meteo call are never cut.

## 5. Minimum Technical Requirements Mapping

| Requirement | Where it lives in BetterRhythm |
|---|---|
| If or Set node | Set "Validate" (step 7) |
| Code node | JavaScript: parse command (2), build schedule (8), update profile (4a) |
| Router / Switch | Switch "Command Router" (step 4) |
| API connection | HTTP Request to Open-Meteo (step 5) |
| Message trigger | Telegram Trigger (step 1), not Chat Message |
| Database | Google Sheets, tabs profile, tasks, signals, errors |
| AI node | LLM Chain "Extractor" (6) and LLM Chain "Writer" (9), OpenRouter free tier |
| Sub-workflow | profile_loader (3) and error_logger (12) |
| Error handling and retries | Retry On Fail x3 on HTTP and AI nodes, JSON defaults, planning without weather, Error Logger fallback reply |

## System Architecture & Workflow Graph

**Every message takes one of three paths, and every failure is caught**




## Connectors and triggers

| Tool | Kind | Role | Auth / cost |
|---|---|---|---|
| Telegram Trigger and Send Message | trigger, output | Starts every run; sends the formatted reply | BotFather token, free |
| Open-Meteo | HTTP API | Hourly forecast for outdoor tasks | Keyless, free |
| OpenRouter models | AI nodes | Extract structure, write the plan | Free tier |

| Tool | Kind | Role | Auth / cost |
| --- | --- | --- | --- |
| Google Sheets | Database | Profile, tasks, signals, errors | Free account |